Laboratório Funcional · Aula 12

Daniel Mitsuo Tsuyama · RA 24160006-2 · Turma ESOFT6SN-B
Gerado em 16/10/2026, 20:00:56 · tempo registrado no cronômetro: 0:00

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

agora é impura porque lê o relógio: com os mesmos parâmetros (nenhum), o resultado muda a cada chamada, então depende de algo de fora. poe é impura porque append altera a própria lista recebida, e isso é um efeito colateral.

Prática 1: Funções puras e formas funcionais

Previsão: O que as linhas 1, 4 e 7 vão imprimir?

Previsão (linhas 1, 4 e 7): A-1 A-2, 14 e [7, 3, 9] [3, 7, 9].

Saídas (original e com compor(g, f))

```
1: A-1 A-2
2: True
3: 8
4: 14
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]
```

1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

proximo_id lê e altera a variável global contador, então o resultado depende do histórico de chamadas e não só do parâmetro. Isso dificulta os testes: é preciso preparar e zerar o estado antes de cada teste, e a ordem em que os testes rodam muda o resultado.

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

compor(f, g)(4) = f(g(4)) = f(12) = 14. compor(g, f)(4) = g(f(4)) = g(6) = 18. A diferença existe porque a composição não é comutativa: a ordem em que as funções são aplicadas muda o resultado.

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

sorted(notas) cria uma lista nova e deixa a original intacta. notas.sort() altera a própria lista. O sorted segue o estilo funcional, porque não tem efeito colateral e só devolve um valor.

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

A

Previsão: (cdr '(A B C))

(B C)

Previsão: (cons '(A B) '(C D))

((A B) C D)

Saídas (original e sem o apóstrofo)

```
22
14
25
8
negativo
A
(B C)
(A B C)
((A B) C D)
B
```

2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo é a abreviação de quote: impede a avaliação e trata (A B C) como dado. Sem ele, o Scheme tentou avaliar (A B C) como uma chamada de função, usando A como função e B e C como parâmetros, e falhou porque A não está definida.

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
Expressão: (car (cdr (cdr '(A B C)))). Resultado: C.
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

n = 6: comum em 12 passos, até 5 multiplicações esperando (expressão de até 42 caracteres); em cauda em 8 passos, nada esperando (até 18 caracteres).

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

No Scheme vai funcionar e imprimir 1000000, porque conta-ate é recursiva em cauda (a chamada recursiva é a última operação) e o Scheme reaproveita o registro de ativação, então a pilha não cresce. No Python, conta_ate(500, 0) funciona, mas conta_ate(1_000_000, 0) deve dar RecursionError, porque o Python não otimiza a chamada em cauda e limita a profundidade da pilha (cerca de 1000 chamadas).

Saída do Scheme

```
#t
#f
2432902008176640000
2432902008176640000
1000000
```

Saída do Python (as últimas linhas bastam)

```
500
Traceback (most recent call last):
  File "/box/main.py", line 9, in <module>
    print(conta_ate(1_000_000, 0))
          ^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
```

3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na recursão comum, cresce a quantidade de multiplicações pendentes: a expressão vira (* 4 (* 3 (* 2 ...))), e cada multiplicação espera a chamada de dentro voltar, ocupando um registro na pilha. Na recursão em cauda não cresce nada, porque o resultado parcial vai no parâmetro acumulador e a chamada recursiva é a última operação.

3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

O Scheme faz a otimização da chamada em cauda: como nada fica esperando, ele reaproveita o registro de ativação e a recursão ocupa espaço constante, como um laço. O Python não faz essa otimização e ainda limita a profundidade da pilha (cerca de 1000 chamadas), então a recursão de um milhão de chamadas estoura com RecursionError, mesmo sendo recursiva em cauda.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

```
tamanho: 3  ok
soma-quadrados: 14  ok
pares: (2 4 6)  ok
```

4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
(define (tamanho lis)
  (if (null? lis)
      0
      (+ 1 (tamanho (cdr lis)))))

(define (soma-quadrados lis)
  (if (null? lis)
      0
      (+ (* (car lis) (car lis)) (soma-quadrados (cdr lis)))))

(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis)) (cons (car lis) (pares (cdr lis))))
        (else (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

o mais difícil foi pensar na função pares, que tem três casos em vez de dois. Em todas as funções o null? é o caso base (lista vazia), o car pega o elemento atual (para contar, elevar ao quadrado ou testar se é par) e o cdr é o que se passa na chamada recursiva, para tratar o resto da lista. Em pares, o cons monta a lista de volta só com os elementos que passam no teste even?.

Prática 5: Haskell: padrões, compreensões e preguiça

Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

take 5 squares deve imprimir [0,1,4,9,16], porque o Haskell usa avaliação preguiçosa: só calcula os 5 primeiros elementos da lista infinita, e o resto nunca é calculado. primeiro 42 (div 1 0) deve imprimir 42, sem erro, porque a função ignora o segundo parâmetro e o Haskell é não estrito, então div 1 0 nunca chega a ser avaliado.

Saídas (original e com member 17)

```
2432902008176640000
[1,2,4,7,14]
```

```
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
42
```

5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

O Haskell usa avaliação preguiçosa: uma expressão só é calculada quando o valor é necessário. take 5 só pede os cinco primeiros elementos de squares, então o resto da lista infinita nunca é calculado. Em primeiro 42 (div 1 0), o segundo parâmetro é ignorado pela função, e como o Haskell é não estrito, div 1 0 nunca é avaliado, então não há erro de divisão por zero.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

Como 17 não é quadrado perfeito, member 17 squares percorre a lista infinita sem nunca encontrar o 17 e sem nunca chegar ao fim, então não termina. Já 16 aparece na lista, e a busca para ao encontrá-lo. Correção: como squares está em ordem crescente, dá para desistir quando o elemento atual passa do valor procurado.

Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

mais10(5) imprime 15; [1, 2, 3].map(vezes(5)) imprime [5, 10, 15].

Saída do JavaScript

```
15
[ 5, 10, 15 ]
56
{ nome: 'Ana', nota: 7 } { nome: 'Ana', nota: 8 }
```

Saída do Java

```
14
18
1050.0
[CABO, MONITOR, MOUSE, TECLADO]
```

6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

É um fechamento: a função devolvida guarda a ligação com o ambiente onde foi criada, e as variáveis capturadas continuam existindo enquanto a função existir. Por isso o n = 10 de somador(10) continua disponível em mais10, mesmo depois de somador ter terminado.

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

f.compose(g) aplica primeiro g e depois f, então f(g(4)) = f(12) = 14. f.andThen(g) aplica primeiro f e depois g, então g(f(4)) = g(6) = 18. O compose corresponde ao compor(f, g) da Prática 1.

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);
        int soma = nums.stream()
            .filter(n -> n % 2 == 0)
            .mapToInt(n -> n * n)
            .sum();
        System.out.println(soma);
    }
}
```

7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
const alunos = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 5.0 },
  { nome: "Carla", nota: 7.0 },
  { nome: "Davi", nota: 9.5 },
];
const aprovados = alunos
  .filter((a) => a.nota >= 6)
  .map((a) => a.nome.toUpperCase());
console.log(aprovados.join(", "));
```

Saídas das suas versões

```
244
ANA, CARLA, DAVI
```

7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

A versão funcional é mais fácil de ler, porque diz o que está sendo feito (filtrar, transformar, somar) sem índice nem variável acumuladora. A versão com laço é mais fácil de depurar, porque dá para colocar um print ou um breakpoint dentro do laço e ver cada valor intermediário. No pipeline é preciso quebrar a cadeia para olhar o meio.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

a versão em Haskell é mais fácil de entender, porque descreve a transformação (gera a lista, eleva ao cubo, soma) sem controlar índice nem acumulador, e fica mais próxima da definição matemática. Aceitaria o custo de cerca de 2 vezes no tempo de execução em sistemas em que correção e facilidade de manutenção pesam mais que a velocidade, como ferramentas de análise de dados ou sistemas financeiros. Não aceitaria em sistemas de tempo real ou de alto desempenho, como jogos ou controle de equipamentos.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

Preguiça do Haskell: Me surpreendi com o Haskell conseguir trabalhar com uma lista infinita e ainda ignorar div 1 0 sem dar erro, graças à avaliação preguiçosa.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

O erro de impressão está na primeira compreensão: b <- h deveria ser b <= h. Com <-, o Haskell entende que h é uma lista da qual b é tirado, mas h também é um elemento da lista t. O compilador então infere um tipo infinito, com uma mensagem parecida com Occurs check: cannot construct the infinite type: a ~ [a] (cole a mensagem real do OneCompiler). Correção: sort [b | b <- t, b <= h].